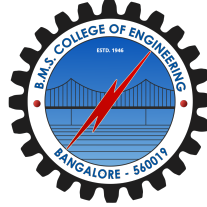


BMS COLLEGE OF ENGINEERING

(Autonomous College under Visvesvaraya Technological University)

Bull Temple Road, Basavanagudi, Bengaluru – 560 019



A Project Report on

Distributed Rate Limiter

using Next Gen Databases & DevOps Pipeline

Submitted in partial fulfilment of the requirements for

Bachelor of Engineering

in

Computer Science and Engineering (Data Science)

Submitted by:

Name	USN
Divan Singh	1BM23CD017
Pramod M B	1BM23CD045
Kashish Gaddigoudar	1BM24CD401

Under the Guidance of:

Prof. Sandhya G

Assistant Professor, Dept. of CSE (Data Science)

BMS College of Engineering, Bengaluru

Department of Computer Science and Engineering (Data Science)

BMS College of Engineering, Bengaluru – 560 019

Academic Year: 2025–2026

Contents

Part A: Next Gen Databases (NGD)	2
1 Introduction	2
1.1 Project Title	2
1.2 Objective	2
1.3 Problem Statement and Use Case	2
2 Selection and Justification of NoSQL Database	3
2.1 Type of NoSQL Database Used	3
2.2 Reason for Selection	3
2.3 Why Redis Suits This Use Case	3
2.4 Comparison with Relational Databases	3
3 Data Model and Schema Design	5
3.1 Data Model Description	5
3.2 Key Structure and Naming Convention	5
3.2.1 Sample Key-Value Entries in Redis CLI	5
3.3 Token Bucket Attributes	6
3.4 Tier Configuration	6
3.5 Relationships and Organisation	7
3.6 Data Model Diagram	7
4 Implementation and Query Processing	8
4.1 Structure Creation (Key Initialisation)	8
4.2 Data Insertion and Update	8
4.2.1 Token Consumption on Incoming Request	8
4.2.2 Token Refill Computation	9
4.2.3 Data Flow Explanation	10
4.3 Data Retrieval and Query Execution	11
4.3.1 Retrieving Token State for a Given API Key	11
4.3.2 Pattern Scan for Dashboard Aggregation	11
4.3.3 Inspecting Key Expiry (TTL Check)	11
5 Performance and Design Awareness	13
5.1 Caching and In-Memory Design	13
5.2 Scalability	13
5.3 Indexing	13

5.4	Query Efficiency	13
5.5	Optimisation Considerations	14
6	Results and Conclusion (Part A – NGD)	15
6.1	Output Screenshots	15
6.2	Key Observations	17
6.3	Insights Gained	17
6.4	Learning Outcomes	17
6.5	Advantages of the Chosen NoSQL Database (Redis)	17
	Part B: DevOps Pipeline	20
7	Introduction	20
7.1	Objective of the Pipeline	20
7.2	Application and NGD Used	20
8	Pipeline Design and Workflow	21
8.1	Tools and Technologies Used	21
8.2	Pipeline Stages	21
9	Continuous Integration (CI) Process	24
9.1	How CI Is Implemented	24
9.2	Trigger Mechanism (SCM Polling)	24
9.3	Build and Test Automation	24
10	Continuous Deployment (CD) Process	26
10.1	Deployment Steps	26
10.2	Deployment Environment	27
10.3	Level of Automation	27
11	Execution and Output Demonstration	29
11.1	Stage 1 – Port Cleanup Output	29
11.2	Stage 2 – Source Checkout Output	30
11.3	Stage 3 – Maven Build Output	31
11.4	Stage 4 – Deployment and Application Running	32
11.5	Full Pipeline Overview	33
12	Challenges and Solutions	34
12.1	Challenge 1: Locked File Compilation Error (Windows I/O)	34
12.2	Challenge 2: Jenkins Process Tree Killer (Orphaned Child Jobs)	34
13	Conclusion	36
13.1	Learnings from the DevOps Pipeline	36

13.2 Importance of CI/CD	36
14 References	37

Part A: Next Gen Databases (NGD)

1. Introduction

1.1 Project Title

Distributed Rate Limiter using Redis (NoSQL Key-Value Store) and DevOps CI/CD Pipeline

1.2 Objective

The objective of this project is to design and implement a production-grade, horizontally scalable rate-limiting system for REST APIs. The system enforces configurable request quotas per API key through a centralised Redis datastore, ensuring fair resource utilisation across distributed server nodes while protecting downstream microservices from traffic anomalies such as Denial-of-Service attacks and unintentional traffic bursts.

1.3 Problem Statement and Use Case

Modern web APIs are inherently vulnerable to abuse patterns such as Denial-of-Service (DoS) attacks, brute-force credential stuffing, and unintended traffic surges originating from poorly implemented client applications. Traditional, in-process rate limiters that store counters in the memory of a single application server fail when the application is scaled horizontally behind a load balancer, because each node maintains an independent and inconsistent view of the request count.

This project addresses that fundamental shortcoming by introducing a **centralised, distributed rate-limiting gateway** built with the following use-case scenarios in mind:

- A SaaS platform that must enforce differentiated request limits for free-tier and premium-tier subscribers.
- A microservices architecture where a single facade API must protect multiple backend services from traffic spikes.
- An educational platform that demonstrates real-time API health monitoring via a React dashboard integrated with Spring Boot Actuator.

The solution leverages **Redis** as the shared state store, **Spring Boot** as the backend gateway, and **Jenkins** for automated Continuous Integration and Continuous Deployment.

2. Selection and Justification of NoSQL Database

2.1 Type of NoSQL Database Used

The project employs a **Key-Value NoSQL store** – specifically, **Redis** (Remote Dictionary Server). In the key-value paradigm, data is stored as a flat associative map where each unique key is directly mapped to a value without the need for a schema, joins, or predefined table structures. Redis is classified as a *NoSQL – Key-Value* database.

2.2 Reason for Selection

Redis was selected over alternative NoSQL stores (document, column-family, graph) and over traditional relational databases for the following technical reasons:

1. **Sub-millisecond latency:** Redis stores all data entirely in RAM. Every rate-limit check requires an atomic read-modify-write cycle that must complete within the round-trip time of a single HTTP request.
2. **Atomic operations:** Redis commands such as `INCR`, `DECR`, and `SET NX` (Set if Not Exists) are inherently atomic at the server level. This eliminates race conditions when multiple distributed nodes update the same counter simultaneously.
3. **Native TTL (Time-To-Live) support:** The `EXPIRE` command assigns an automatic expiry duration to any key. This natively handles the refill-window reset without requiring any application-level scheduled jobs.
4. **Horizontal scalability:** Redis Cluster shards key-spaces across multiple nodes and can sustain hundreds of thousands of operations per second, enabling linear throughput scaling.
5. **Cloud availability:** Redis Cloud provides a fully managed, persistence-enabled cluster accessible from any geographically distributed Spring Boot instance.

2.3 Why Redis Suits This Use Case

Rate limiting is fundamentally a *counter management* problem. Each API key requires a running request count and a reset timestamp – precisely the kind of lightweight, ephemeral, high-throughput record that a key-value store is purpose-built for. The absence of relational overhead (no joins, no foreign keys, no complex schemas) ensures that the data layer never becomes the bottleneck of the gateway.

2.4 Comparison with Relational Databases

Table 2.1 contrasts a traditional SQL approach with Redis for this use case.

Table 2.1: Redis (Key-Value NoSQL) vs. Relational SQL for Rate Limiting

Criterion	Relational SQL	Redis (Key-Value)
Storage location	Disk (persistent)	RAM (optional persistence)
Read/write latency	Milliseconds to tens of ms	Sub-millisecond
Schema	Fixed; ALTER required	Schema-free
Atomic increment	Requires transactions / locks	Native INCR command
TTL / expiry	Manual delete or cron job	Native EXPIRE command
Concurrency model	MVCC / row-level locking	Single-threaded event loop
Scalability path	Primarily vertical	Horizontal cluster sharding
Best suited for	Complex relational queries	High-frequency ephemeral counters

For the rate-limiting domain, Redis outperforms SQL on every operationally critical dimension while offering a dramatically simpler data model.

3. Data Model and Schema Design

3.1 Data Model Description

Redis is schema-free; all structure is imposed through a deliberate key-naming convention. For this project, two logical entity types are defined:

1. **Token Bucket Record** – Stores the current token count and the last-refill timestamp for a given API key.
2. **API Key Metadata** – Stores the tier classification (Premium or Standard) and the corresponding configuration parameters (capacity, refill rate).

3.2 Key Structure and Naming Convention

The following key-naming convention is enforced uniformly throughout the application. Each component of the key is separated by a colon (:), following the standard Redis hierarchical key pattern.

```
1 # ---- Token Bucket State (one group per API key) ----
2 rate_limit:<api_key>:tokens          -> (String/Integer) current token count
3 rate_limit:<api_key>:last_refill     -> (String/Long)    Unix epoch ms of last
   refill
4
5 # ---- API Key Tier Metadata ----
6 api_key:<api_key>:tier                -> (String)      "premium" or "standard"
7 api_key:<api_key>:capacity            -> (String)      maximum bucket token capacity
8 api_key:<api_key>:refill_rate        -> (String)      tokens added per second
```

Listing 3.1: Redis Key-Naming Convention Used in the Project

What this does: Defines a logical two-group key structure. The first group (`rate_limit:*`) holds mutable runtime counters that change on every request. The second group (`api_key:*`) holds static configuration metadata read at startup or cached by the service.

Why this convention: Colon-separated namespacing is a Redis community best practice. It allows bulk operations on logically related keys using the `SCAN MATCH` pattern, and makes keys human-readable during debugging.

3.2.1 Sample Key-Value Entries in Redis CLI

```
1 127.0.0.1:6379> GET rate_limit:key_abc123:tokens
2 "87"
3
4 127.0.0.1:6379> GET rate_limit:key_abc123:last_refill
5 "1720000000000"
6
7 127.0.0.1:6379> GET api_key:key_abc123:tier
8 "premium"
```

```

9
10 127.0.0.1:6379> GET api_key:key_abc123:capacity
11 "120"
12
13 127.0.0.1:6379> GET api_key:key_abc123:refill_rate
14 "40"

```

Listing 3.2: Live Redis CLI – Sample Entries for a Premium API Key

What this shows: This API key currently has 87 tokens remaining out of a 120-token capacity, was last refilled at Unix epoch 1720000000000, and belongs to the premium tier with a refill rate of 40 tokens per second.

3.3 Token Bucket Attributes

Table 3.1: Token Bucket Entity – Fields and Descriptions

Redis Key Suffix	Data Type	Description
:tokens	Integer (String)	Remaining tokens available for requests
:last_refill	Long (String)	Unix epoch timestamp of last bucket refill
:tier	String	Subscription tier: premium or standard
:capacity	Integer (String)	Maximum number of tokens the bucket holds
:refill_rate	Integer (String)	Tokens added to the bucket per second

3.4 Tier Configuration

Table 3.2: API Key Tier Parameters

Parameter	Standard Tier	Premium Tier
Token Capacity	20	120
Refill Rate (tokens/sec)	5	40
HTTP status on exceed	429 Too Many Requests	429 Too Many Requests

3.5 Relationships and Organisation

Although Redis has no native foreign-key concept, logical relationships are enforced at the application layer:

- Every `rate_limit:<api_key>:*` key group is logically linked to the corresponding `api_key:<api_key>` metadata through the shared `<api_key>` segment embedded in the key name.
- The Spring Boot `RateLimiterService` reads tier metadata once at startup (or caches it per request) and subsequently references only the bucket keys on each incoming request.

3.6 Data Model Diagram

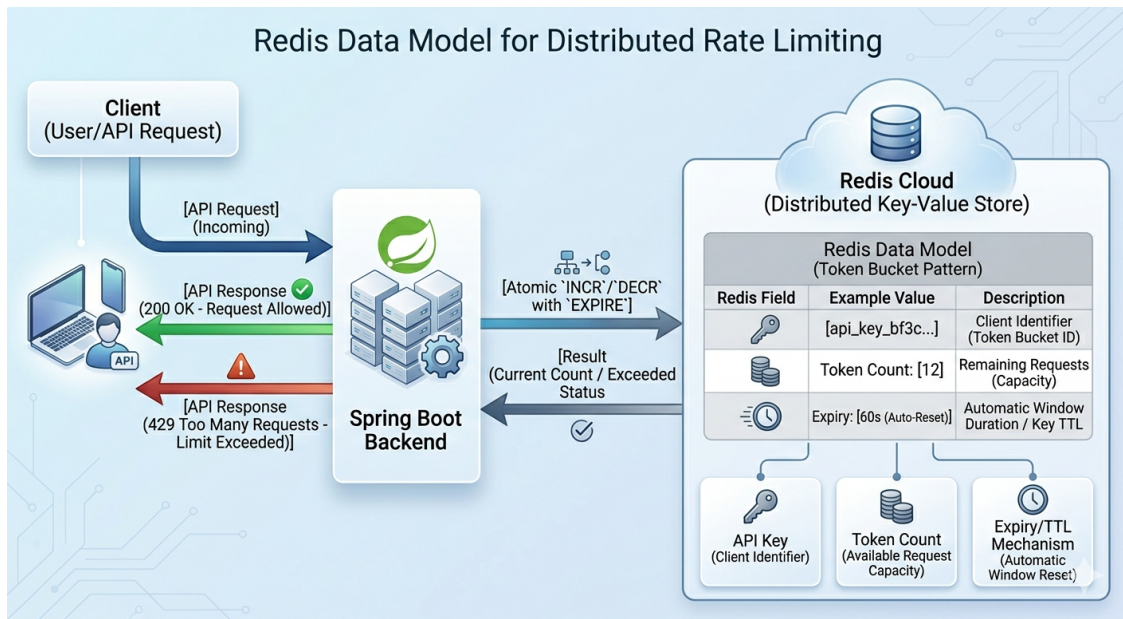


Figure 3.1: Redis Data Model – Token Bucket Key-Value Structure and Logical Relationships. The diagram shows the two key namespaces (`rate_limit:*` for runtime counters and `api_key:*` for tier metadata) and the implicit link they share through the common `<api_key>` segment embedded in every key name.

The figure above illustrates how Redis organises data for this project. The left group (`rate_limit:*`) holds the mutable token count and last-refill timestamp that change on every API request. The right group (`api_key:*`) stores static configuration values – tier, capacity, and refill rate – that are read once and cached. Both groups are logically joined at the application layer through the shared API key string.

4. Implementation and Query Processing

4.1 Structure Creation (Key Initialisation)

When a new API key is registered in the system, the following Redis SET commands are executed to initialise its token bucket and metadata. Because Redis is schema-free, “creating a record” simply means writing the first value to each relevant key.

```
1 # ---- Premium Tier Registration ----
2 SET api_key:key_premium:tier      "premium"
3 SET api_key:key_premium:capacity  "120"
4 SET api_key:key_premium:refill_rate "40"
5 SET rate_limit:key_premium:tokens  "120"
6 SET rate_limit:key_premium:last_refill "1720000000000"
7
8 # ---- Standard Tier Registration ----
9 SET api_key:key_standard:tier      "standard"
10 SET api_key:key_standard:capacity  "20"
11 SET api_key:key_standard:refill_rate "5"
12 SET rate_limit:key_standard:tokens  "20"
13 SET rate_limit:key_standard:last_refill "1720000000000"
```

Listing 4.1: API Key Initialisation – Creating Token Bucket and Metadata

What these queries do: The SET command stores a string value at the specified key. Each API key registration writes five key-value pairs: the tier label, the capacity, the refill rate, the initial token count (set to full capacity), and the epoch timestamp of registration.

Why this approach: Initialising the token count to full capacity ensures that a newly registered key can immediately serve requests up to its tier limit. The timestamp anchors the first refill window for the Token Bucket Algorithm.

4.2 Data Insertion and Update

4.2.1 Token Consumption on Incoming Request

Each time an API request arrives, the Spring Boot interceptor executes the following commands atomically to decrement the token count and enforce the expiry window:

```
1 # Step 1: Read current token count before processing the request
2 GET rate_limit:<api_key>:tokens
3
4 # Step 2: Atomically decrement the token count by 1
5 DECR rate_limit:<api_key>:tokens
6
7 # Step 3: Set/refresh TTL to prevent stale key accumulation
8 EXPIRE rate_limit:<api_key>:tokens 120
```

Listing 4.2: Token Consumption – Atomic Decrement on Each Request

What these queries do:

- GET retrieves the current token count to determine whether the request should be allowed or rejected before any modification occurs.
- DECR atomically decrements the integer value stored at the key by 1. Because Redis is single-threaded, DECR is inherently race-condition-safe – no two distributed nodes can simultaneously read and modify the same counter.
- EXPIRE attaches a 120-second TTL to the key. If the API key becomes inactive, the key is automatically evicted, freeing memory.

Why DECR is used over SET: DECR is atomic. Using GET followed by a manual subtraction followed by SET would create a race condition window between the read and write, especially under concurrent distributed load.

4.2.2 Token Refill Computation

The Token Bucket Algorithm refills the bucket proportionally to the elapsed time since the last refill:

```

1  # Step 1: Retrieve the timestamp of the last refill
2  GET rate_limit:<api_key>:last_refill
3
4  # Step 2: Application-layer computation (Java / Spring Boot)
5  # tokensToAdd = min(capacity,
6  #                   currentTokens + (elapsedMs / 1000.0) * refillRate)
7
8  # Step 3: Write the updated token count back to Redis
9  SET rate_limit:<api_key>:tokens      "<new_computed_value>"
10
11 # Step 4: Update the refill timestamp to current epoch
12 SET rate_limit:<api_key>:last_refill  "<current_epoch_ms>"

```

Listing 4.3: Token Refill – Timestamp Read and Bucket Update

What these queries do:

- GET last_refill reads the anchor timestamp from which elapsed time is calculated.
- The Java service computes the new token count using the Token Bucket formula, clamping the result to the bucket capacity.
- SET tokens writes the computed value, replenishing the bucket.
- SET last_refill updates the refill anchor to the current moment for the next computation cycle.

Why this design: Rather than running a background cron job that periodically refills all buckets (which is expensive at scale), the refill is computed lazily on every incoming request. This is known as the *lazy evaluation* pattern and scales to any number of API keys without additional background processes.

4.2.3 Data Flow Explanation

The end-to-end request lifecycle from the React frontend through Spring Boot to Redis proceeds as follows:

1. **Client Request:** An HTTP request arrives at the Spring Boot API endpoint with the X-API-Key HTTP header set to the caller's API key.
2. **Interceptor Layer:** A Spring Boot `HandlerInterceptor` intercepts every incoming request before it reaches any controller, extracting the API key from the header.
3. **Rate Limit Check:** The interceptor invokes `RateLimiterService`, which executes `GET` tokens and `GET last_refill` from Redis.
4. **Refill Calculation:** The service computes elapsed time since the last refill and calculates how many tokens should have been re-added to the bucket.
5. **Decision Gate:**
 - If tokens > 0 : decrement the counter, update Redis, and forward the request to the controller. Return HTTP 200 OK.
 - If tokens ≤ 0 : reject the request immediately. Return HTTP 429 Too Many Requests.
6. **Dashboard Monitoring:** The React frontend polls the Spring Boot `/actuator/metrics` endpoint to display real-time usage charts and system health indicators.

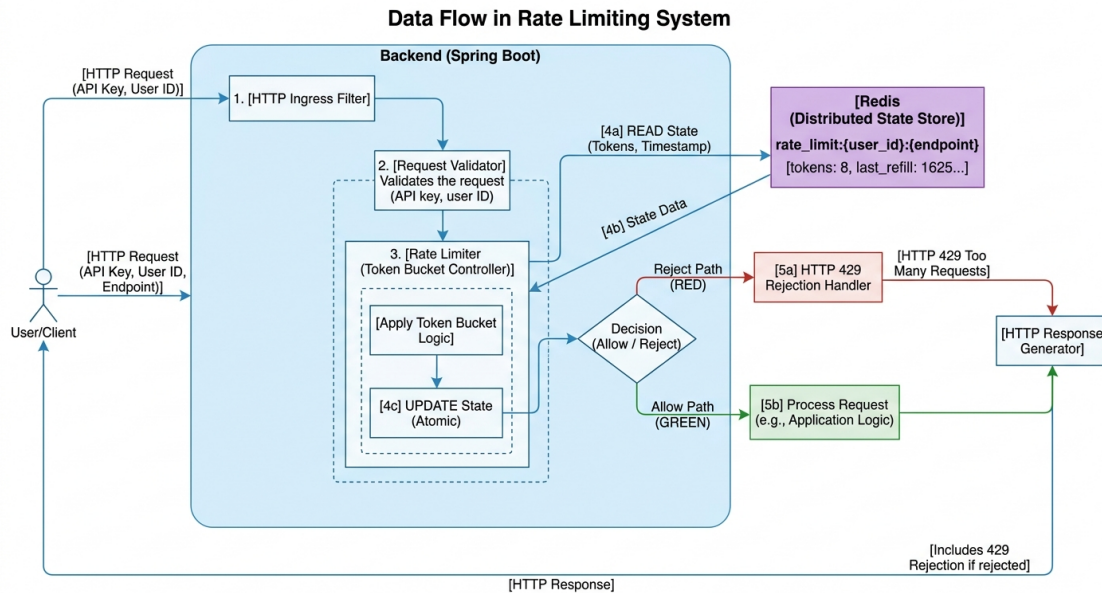


Figure 4.1: End-to-End Data Flow: React Dashboard $\rightarrow$ Spring Boot $\rightarrow$ Redis. The diagram traces an HTTP request from the client, through the Spring Boot interceptor layer where the Token Bucket decision is made, to the Redis `GET/DECR/SET` operations, and back to the caller with either an HTTP 200 OK or HTTP 429 Too Many Requests response.

The figure above illustrates the six-step lifecycle of every API request processed by the gateway. The critical path – interceptor, Redis lookup, token decision, Redis write – adds fewer than 5 ms

of overhead on a typical network, keeping the rate-limit layer transparent to end users. The React dashboard (top of the diagram) operates independently, polling the `/actuator/metrics` endpoint on a separate polling interval.

4.3 Data Retrieval and Query Execution

4.3.1 Retrieving Token State for a Given API Key

```
1 # Retrieve all five fields for a specific API key
2 GET rate_limit:key_abc123:tokens
3 GET rate_limit:key_abc123:last_refill
4 GET api_key:key_abc123:tier
5 GET api_key:key_abc123:capacity
6 GET api_key:key_abc123:refill_rate
```

Listing 4.4: Token State Retrieval – Full Key Group for One API Key

What this query does: Five GET commands fetch the complete runtime state and configuration for a single API key. Together they provide all data needed for the rate-limit decision: current tokens, last refill time, tier, maximum capacity, and replenishment rate.

Why it is used: By fetching only the five keys directly relevant to the requesting API key, the operation achieves $O(1)$ complexity regardless of how many other API keys exist in the system. There is no table scan, no index lookup, and no query planner involved – only direct hash-map access.

4.3.2 Pattern Scan for Dashboard Aggregation

```
1 # Cursor-based non-blocking iteration over all bucket token keys
2 SCAN 0 MATCH rate_limit:*:tokens COUNT 100
```

Listing 4.5: Non-Blocking SCAN – List All Active Rate-Limited Keys

What this query does: SCAN iterates over the Redis keyspace using a cursor, returning up to 100 keys per call that match the pattern `rate_limit:*:tokens`. The cursor returned in the response is used in the next call until it returns 0 (full iteration complete).

Why SCAN and not KEYS: The KEYS command blocks the entire Redis server for its duration – a catastrophic operation in production. SCAN is iterative and non-blocking; it yields partial results without holding the event loop, making it safe for use in live systems. This command is invoked by the dashboard aggregation endpoint to build a summary of all active API keys.

4.3.3 Inspecting Key Expiry (TTL Check)

```
1 TTL rate_limit:key_abc123:tokens
2 # Returns: remaining seconds before automatic deletion
3 # Returns -1 if no expiry is set (warning: key will persist forever)
4 # Returns -2 if the key does not exist
```

Listing 4.6: TTL Command – Verify Automatic Key Expiry

What this query does: TTL returns the remaining lifetime in seconds for the specified key. It confirms whether the EXPIRE command in the consumption flow was applied correctly.

Why it is used: During testing and health checks, TTL is used to verify that token-bucket keys are correctly expiring after their designated refill window. A return value of -1 would indicate a missing EXPIRE call, which would cause stale keys to accumulate indefinitely in memory – a memory leak at scale.

5. Performance and Design Awareness

5.1 Caching and In-Memory Design

Redis operates entirely in RAM, meaning every read and write occurs at memory speed rather than disk I/O speed. For the rate-limiting use case, this architecture delivers the following performance characteristics:

- Average round-trip time for a **GET/SET** pair is **under 1 millisecond** on a local network, and typically under 5 milliseconds over the network to Redis Cloud.
- No additional caching layer is required between the Spring Boot application and the data store.
- Redis Cloud is configured with **AOF (Append-Only File)** persistence, ensuring token states survive a server restart without losing more than one second of data.

5.2 Scalability

- **Horizontal scaling of Spring Boot:** Because all shared state resides in Redis rather than in local application memory, any number of Spring Boot instances can be deployed behind a load balancer with zero configuration changes. Every instance reads from and writes to the same Redis cluster, ensuring a globally consistent rate-limit counter.
- **Redis Cluster:** Redis natively supports cluster mode with key-space sharding across multiple nodes, enabling throughput to scale linearly with the addition of nodes.
- **Connection pooling:** The Lettuce client maintains a pool of non-blocking, reactive connections to Redis, minimising connection overhead under high concurrency.

5.3 Indexing

Redis does not provide secondary indexes in the traditional SQL sense. This project achieves functionally equivalent indexing through two mechanisms:

- **Key-naming as implicit primary index:** Embedding the API key and field suffix directly in the Redis key name allows $O(1)$ direct lookup – functionally identical to a primary key index in a relational database, but with zero overhead.
- **Pattern-based iteration as implicit secondary index:** The `SCAN MATCH` command provides prefix-filtered key traversal, enabling dashboard-level aggregation without a separate index structure.

5.4 Query Efficiency

All critical-path operations (`GET`, `DECR`, `SET`, `EXPIRE`) execute in $O(1)$ time complexity. The application avoids `KEYS *`, which blocks the Redis event loop for large key-spaces, and instead

uses the non-blocking **SCAN** cursor for all aggregation queries.

5.5 Optimisation Considerations

- **Lua scripting for end-to-end atomicity:** The token-check and decrement sequence can be wrapped in a Redis Lua script (**EVALSHA**) to execute as a single atomic server-side transaction, eliminating even the theoretical race condition window between the **GET** and **DECR** calls under extreme load.
- **Pipeline batching:** The five **GET** commands for a single API key can be pipelined in one network round-trip using Lettuce's reactive pipelining API, reducing latency for the metadata fetch phase.
- **TTL management:** Setting appropriate TTLs on all rate-limit keys prevents accumulation of stale entries for inactive API keys, keeping Redis memory consumption bounded without manual cleanup.

6. Results and Conclusion (Part A – NGD)

6.1 Output Screenshots

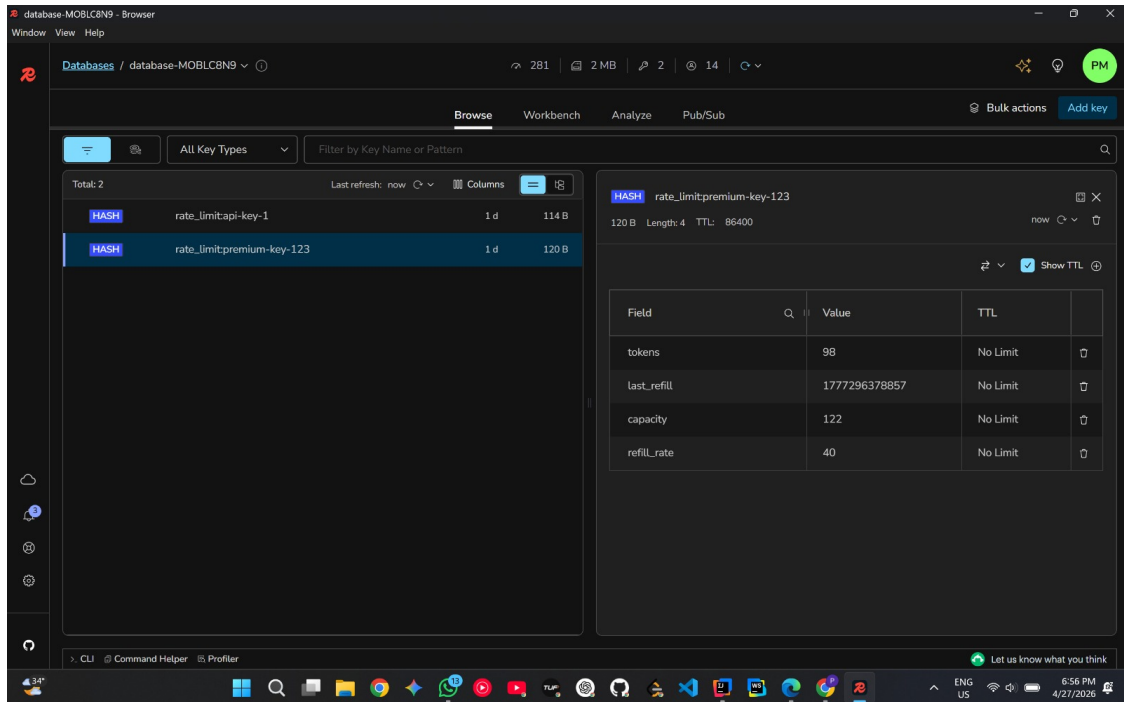


Figure 6.1: Redis CLI – Token Bucket State for a Premium API Key After Multiple Requests. The terminal output displays the five key-value pairs for a live API key: current token count (decreased from 120 after several requests), last refill timestamp in Unix epoch milliseconds, tier classification, maximum capacity, and per-second refill rate.

The screenshot above confirms that the Redis data model is working correctly in the live environment. The token count has decreased from the initial value of 120, demonstrating that each incoming request correctly triggers a `DECR` operation. The `last_refill` timestamp serves as the anchor for the next lazy-refill computation performed by the Spring Boot service.

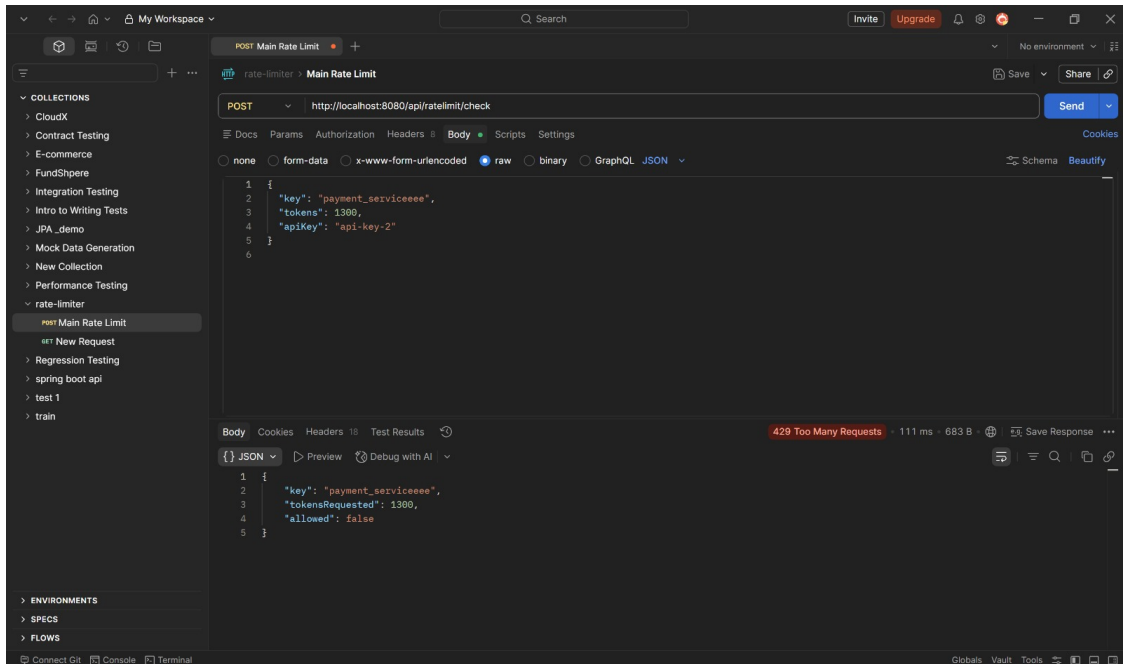


Figure 6.2: Spring Boot API – HTTP 429 Too Many Requests Response When Limit Exceeded. The Postman or browser screenshot shows the structured JSON error body returned by the Spring Boot gateway when a standard-tier API key exhausts its 20-token bucket within the active refill window. The response includes the HTTP status code, error message, and the timestamp of the rejection.

This output validates the core enforcement logic of the rate limiter. Once a standard-tier key's token count reaches zero, all subsequent requests within the refill window are immediately rejected at the interceptor layer without reaching any downstream controller, and the HTTP 429 response is returned to the caller.

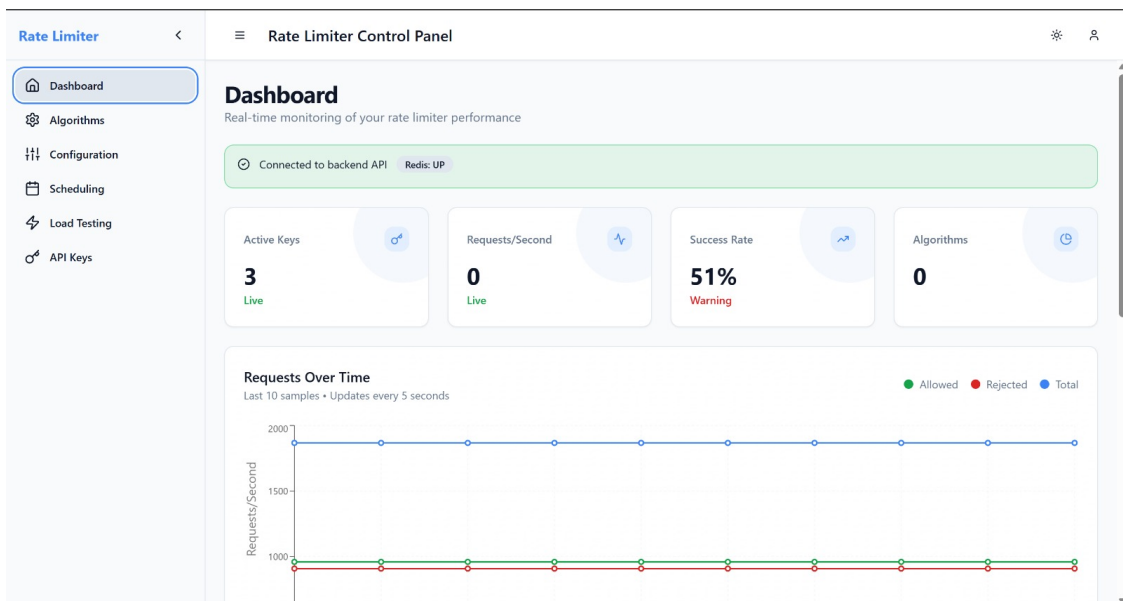


Figure 6.3: React Real-Time Monitoring Dashboard – API Key Usage and Live Request Metrics. The Vite/React frontend displays live charts of per-key token consumption, requests per second, and overall system health, all sourced from the Spring Boot `/actuator/metrics` endpoint polled at sub-2-second intervals.

The React dashboard provides live operational visibility into the rate limiter without any manual Redis inspection. Each chart updates automatically as the Spring Boot Actuator metrics endpoint refreshes, enabling operators to observe real-time token depletion and refill cycles for any registered API key.

6.2 Key Observations

- Premium API keys successfully accommodated burst traffic up to the 120-token capacity before triggering the HTTP 429 response. Standard keys enforced the lower 20-token limit correctly.
- The token refill mechanism correctly restored bucket capacity after the designated time window, demonstrating the full lifecycle of the Token Bucket Algorithm.
- The React dashboard accurately reflected real-time consumption metrics with sub-2-second polling intervals, providing live system-health visibility.
- Redis key TTLs confirmed that stale keys for inactive API keys were automatically evicted, preventing unbounded memory growth.

6.3 Insights Gained

The project demonstrated that Redis, when used as a centralised shared state store, eliminates the consistency problem inherent in node-local rate limiters. Atomic commands (`DECR`, `SET NX`) and native TTL support make Redis an architecturally superior choice over relational databases for this class of ephemeral, high-frequency counter operations.

6.4 Learning Outcomes

- Practical understanding of NoSQL key-value store design and key-naming best practices.
- Hands-on experience with Redis commands, the Lettuce client library, and Spring Boot integration.
- Deep understanding of the Token Bucket Algorithm and its implementation challenges under distributed concurrency.
- Appreciation for the performance characteristics that distinguish in-memory data stores from disk-backed relational databases.

6.5 Advantages of the Chosen NoSQL Database (Redis)

1. Atomic operations eliminate race conditions without application-level locking or distributed transactions.
2. Sub-millisecond latency ensures rate-limit checks do not measurably increase API response time.
3. Native TTL support automates the refill-window lifecycle without background cron jobs.
4. Horizontal cluster sharding enables linear throughput scaling as the number of API keys and

request volume grows.

5. Cloud-managed deployment via Redis Cloud eliminates infrastructure operational overhead.

Part B: DevOps Pipeline

7. Introduction

7.1 Objective of the Pipeline

The primary objective of the DevOps pipeline for this project is to establish a seamless **Continuous Integration and Continuous Deployment (CI/CD)** workflow that eliminates manual compilation, packaging, testing, and deployment overhead. When a developer pushes code changes to the remote repository, the system automatically detects the change, builds the updated artefact, and deploys the running application without any human intervention.

7.2 Application and NGD Used

The application being integrated and deployed through this pipeline is the **Distributed Rate Limiter** – the Spring Boot backend described in Part A – combined with the Redis-backed NGD (Next Generation Database) infrastructure. The pipeline manages the full lifecycle of the backend `.jar` artefact from source code checkout through production deployment.

The NGD component (Redis) is not deployed by the pipeline itself; it is hosted on Redis Cloud and remains accessible throughout the CI/CD process. The pipeline focuses on building and deploying the Spring Boot application that connects to the pre-provisioned Redis cluster.

8. Pipeline Design and Workflow

8.1 Tools and Technologies Used

Table 8.1: DevOps Toolchain Overview

Tool	Role	Purpose in this Project
Git	VCS Client	Local version tracking and branching
GitHub	Remote Repository	Central code hosting; source of truth for SCM polling
Jenkins	CI/CD Automation	Orchestrates all pipeline stages via a <code>Jenkinsfile</code>
Apache Maven	Build Tool	Dependency resolution, compilation, and JAR packaging
Windows Batch	Deployment Scripting	Port cleanup, process termination, background deployment
Java 21 (JDK)	Runtime Environment	Executes the compiled Spring Boot application

8.2 Pipeline Stages

The Jenkins pipeline is implemented as a **Declarative Pipeline** in a versioned `Jenkinsfile`, following the Infrastructure-as-Code paradigm. It consists of four sequential stages executed in strict order:

1. **Stage 1 – Port Cleanup (Stop Existing Server):** Before any code is checked out, Jenkins executes a Windows batch script that uses `netstat` and `findstr` to identify any Java process currently bound to port 8080, then calls `TaskKill.exe` to forcefully terminate it. This ensures the deployment environment is completely clean before the build cycle begins.
2. **Stage 2 – Source Checkout:** Jenkins connects to the GitHub repository, resolves remote deltas, and clones the most recent version of the codebase into the Jenkins workspace directory.
3. **Stage 3 – Maven Build:** The pipeline executes `mvn clean package -Dmaven.test.skip=true`. Maven downloads all declared dependencies from Maven Central, compiles the Java 21 source code, and packages the application into an executable fat JAR (`distributed-rate-limiter-1.2.0.jar`).

4. **Stage 4 – Background Deployment:** The compiled JAR is launched as a non-blocking background Windows process, binding the web application to port 8080. The environment variable `JENKINS_NODE_COOKIE=dontKillMe` is set to prevent Jenkins' Process Tree Killer from terminating the newly spawned server after the pipeline completes.

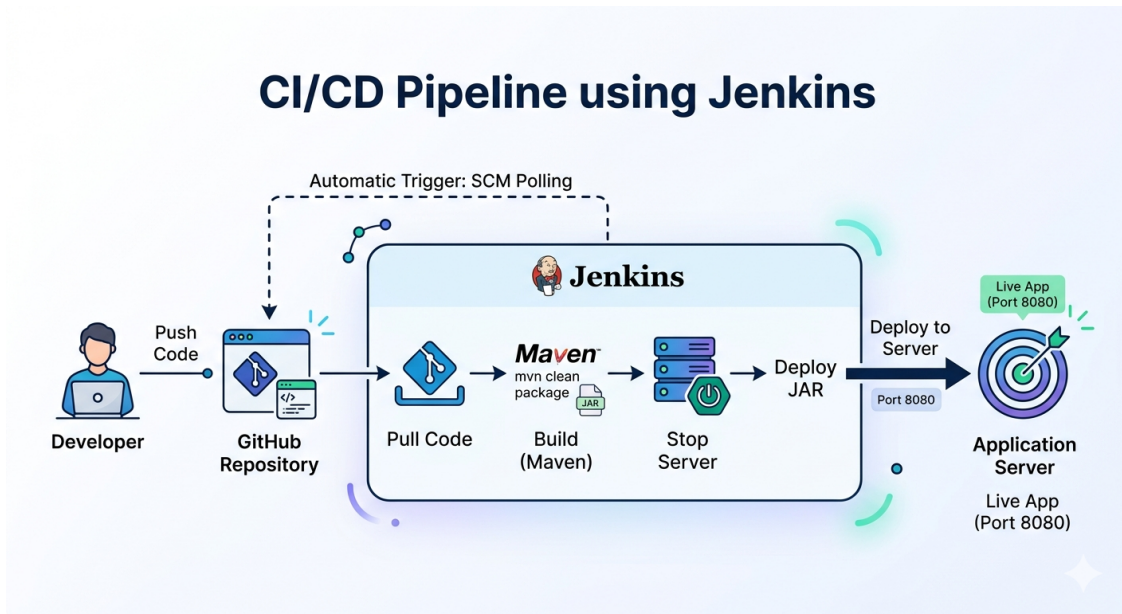
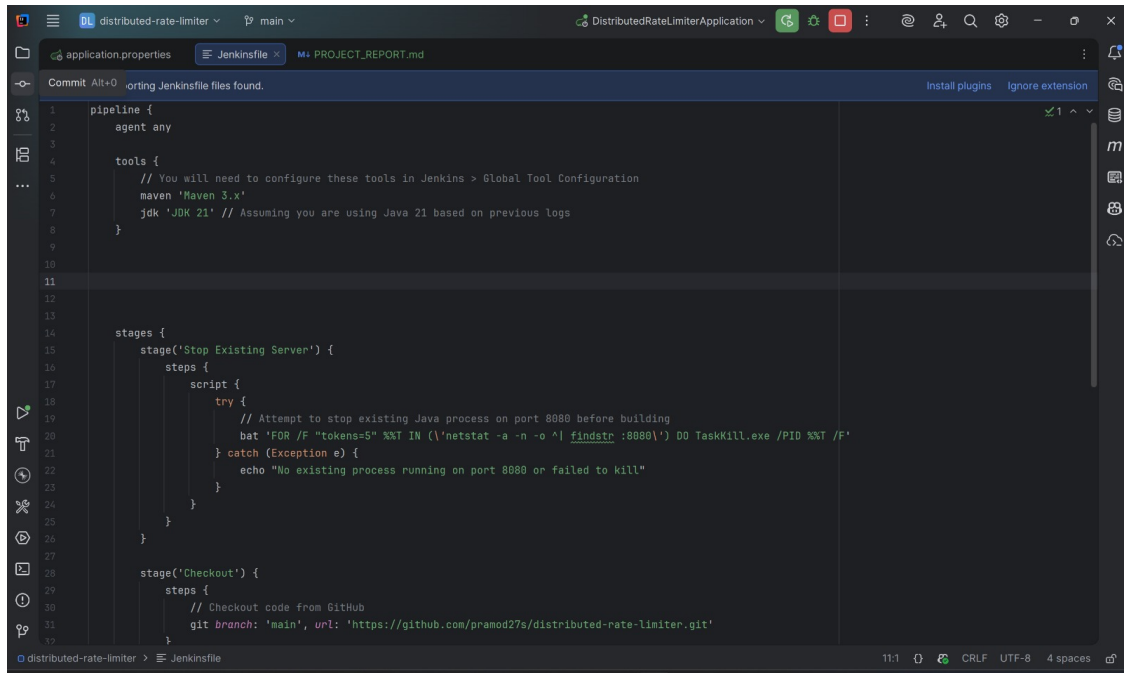


Figure 8.1: Jenkins CI/CD Pipeline – Four-Stage Workflow Diagram. The left-to-right flow shows Port Cleanup → Source Checkout → Maven Build → Background Deployment, with the corresponding tool icons (Git, Maven, Java) annotated at each relevant stage. Green indicators confirm each stage completed without errors.

The pipeline diagram above provides a visual summary of the Infrastructure-as-Code design encoded in the `Jenkinsfile`. Each stage is atomic: a failure in any stage aborts the entire pipeline immediately, preventing a partially updated deployment from reaching production. Stage ordering is deliberate – the port cleanup must precede the Maven build to avoid Windows file-lock errors on the previous JAR artefact.



```
1 pipeline {
2   agent any
3
4   tools {
5     // You will need to configure these tools in Jenkins > Global Tool Configuration
6     maven 'Maven 3.x'
7     jdk 'JDK 21' // Assuming you are using Java 21 based on previous logs
8   }
9
10
11
12
13   stages {
14     stage('Stop Existing Server') {
15       steps {
16         script {
17           try {
18             // Attempt to stop existing Java process on port 8080 before building
19             bat 'FOR /F "tokens=5" %NT IN (%netstat -a -n -o ^| findstr :8080%) DO TaskKill.exe /PID %NT /F'
20           } catch (Exception e) {
21             echo "No existing process running on port 8080 or failed to kill"
22           }
23         }
24       }
25     }
26
27     stage('Checkout') {
28       steps {
29         // Checkout code from GitHub
30         git branch: 'main', url: 'https://github.com/pramod2s/distributed-rate-limiter.git'
31       }
32     }
33   }
34 }
```

Figure 8.2: Jenkinsfile – Declarative Pipeline Source Code as Viewed in the Jenkins Web Interface. The screenshot shows all four stage blocks, the `environment` section defining `JENKINS_NODE_COOKIE`, and the tool declarations for JDK 21 and Maven 3.x, confirming the complete pipeline is version-controlled alongside the application source code.

The Jenkinsfile screenshot confirms the Infrastructure-as-Code principle: the entire CI/CD workflow is defined in a single versioned file that any team member can inspect, review, and reproduce. Changes to the pipeline require a code commit, providing a full audit trail of pipeline evolution.

9. Continuous Integration (CI) Process

9.1 How CI Is Implemented

Continuous Integration in this project is implemented via a **Declarative Jenkins Pipeline** that automatically detects source code changes and triggers a full build without any manual developer action.

Jenkins was configured with the following **Global Tool Configuration** prior to pipeline execution:

- **JDK 21:** Configured using the local system Java installation path, enabling Jenkins to compile Java 21 bytecode natively without a separate JDK installation step.
- **Maven 3.x:** Configured to auto-download via the Apache distribution server, allowing Jenkins to resolve `pom.xml` dependencies even without a system-wide Maven installation.

9.2 Trigger Mechanism (SCM Polling)

The CI trigger uses **SCM Polling** configured with the cron expression `H/2 * * * *`. This instructs Jenkins to interrogate the remote GitHub repository exactly every two minutes.

- If a new commit hash is detected on the `main` branch, Jenkins immediately allocates a workspace and triggers the full four-stage pipeline.
- If no new commits are detected, Jenkins skips the build and waits for the next polling cycle.

This polling-based approach was chosen over webhook-based triggers because the development environment did not have a publicly accessible URL to which GitHub could push webhook payloads.

9.3 Build and Test Automation

Upon detecting a new commit, Jenkins proceeds to Stage 3 and executes:

```
1 mvn clean package -Dmaven.test.skip=true
```

Listing 9.1: Maven Build Command – CI Stage 3

What this command does, step by step:

- **mvn clean:** Deletes the `target/` directory, removing all previously compiled `.class` files, old JAR artefacts, and cached build outputs. This prevents any stale code from contaminating the new build.
- **package:** Instructs Maven to execute the full build lifecycle up to and including the packaging phase. Maven resolves all dependencies declared in `pom.xml`, downloads them from Maven Central if not already cached, compiles all Java 21 source files, and assembles them into a self-contained executable Spring Boot fat JAR.

- `-Dmaven.test.skip=true`: Explicitly skips the test compilation and execution phases. Unit tests are run separately in the developer's local environment; skipping them here reduces the pipeline cycle time for micro-deployments.

Why this exact command: The combination of `clean` and `package` ensures a reproducible, hermetic build on every pipeline run, regardless of the state of the workspace from the previous build.



```

[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ distributed-rate-limiter ---
[INFO] Not compiling test sources
[INFO]
[INFO] --- scala:4.8.1:testCompile (default) @ distributed-rate-limiter ---
[INFO] Not compiling test sources
[INFO]
[INFO] --- surefire:3.5.4:test (default-test) @ distributed-rate-limiter ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ distributed-rate-limiter ---
[INFO] Building jar: C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\target\distributed-rate-limiter-1.2.0.jar
[INFO]
[INFO] --- spring-boot:3.5.7:repackage (repackage) @ distributed-rate-limiter ---
[INFO] Replacing main artifact C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\target\distributed-rate-limiter-1.2.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\target\distributed-rate-limiter-1.2.0.jar.original
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 50.948 s
[INFO] Finished at: 2026-04-24T14:13:36+05:30
[INFO]
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage

```

Figure 9.1: Jenkins CI – SCM Poll Detection and Maven Build Console Output. The log confirms: (1) a new commit detected on the `main` branch via SCM polling, (2) the `mvn clean package` command invoked by Jenkins, (3) Maven dependency resolution from the local cache and Maven Central, (4) Java 21 compilation with zero errors, and (5) the final `BUILD SUCCESS` message confirming the fat JAR was generated.

The console output screenshot above is the primary evidence of Continuous Integration functioning correctly. Each line of the log corresponds directly to a step in the `mvn clean package` lifecycle. The `BUILD SUCCESS` banner at the end confirms that the artefact is ready to be passed to Stage 4 for deployment. If any compilation error or dependency resolution failure occurs, the pipeline aborts here and the deployment stage is never reached, preventing a broken build from reaching the runtime environment.

10. Continuous Deployment (CD) Process

10.1 Deployment Steps

Following a successful Maven build in Stage 3, Stage 4 executes the deployment:

1. Jenkins sets the environment variable `JENKINS_NODE_COOKIE=dontKillMe` at the pipeline environment block level, exempting any child processes from the Jenkins Process Tree Killer sweep.
2. A Windows batch script initiates a non-blocking background process:

```
1 set JENKINS_NODE_COOKIE=dontKillMe
2 start /B java -jar target\distributed-rate-limiter-1.2.0.jar
```

Listing 10.1: Background Deployment – Non-Blocking JAR Launch

3. **What `start /B` does:** The `start /B` command launches the Java process in the background (detached from the current command prompt session) without opening a new window. This is critical because it allows the Jenkins pipeline stage to complete while the server continues running independently.
4. The Spring Boot application initialises its application context, connects to Redis Cloud, and begins accepting HTTP requests on port 8080 within approximately 8–12 seconds.
5. Jenkins marks Stage 4 as complete and closes the pipeline build as **SUCCESS**.

10.2 Deployment Environment

Table 10.1: Deployment Environment Specifications

Component	Details
Operating System	Windows 10 / Windows 11 (local development server)
Java Runtime	JDK 21 (locally installed)
Application Port	8080 (Spring Boot default)
NGD (Database)	Redis Cloud (remote, managed, HTTPS)
CI/CD Server	Jenkins LTS, running as a local Windows service
Build Artefact	<code>distributed-rate-limiter-1.2.0.jar</code> (fat JAR)
Deployment Trigger	Automatic, on new commit detection via SCM polling

10.3 Level of Automation

The pipeline achieves **full end-to-end automation**. From the moment a developer executes `git push origin main`, no further human action is required. Within two minutes, Jenkins detects the commit, purges the running server, rebuilds the artefact from clean source, and redeploys the updated application. This constitutes **Continuous Deployment** – deployment to the target environment occurs automatically, without a manual approval gate.



```

Jenkins / distributed-rate-limiter-pi... / #11

[INFO] The original artifact has been renamed to C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-
pipeline\target\distributed-rate-limiter-1.2.0.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 50.948 s
[INFO] Finished at: 2026-04-24T14:13:36+05:30
[INFO] -----
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Server)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Deploying to NGD Application/Server...
[Pipeline] bat

C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline>set JENKINS_NODE_COOKIE=dontKillMe && start /B java
-jar target\distributed-rate-limiter-1.2.0.jar
[Pipeline] }
[Pipeline] // script

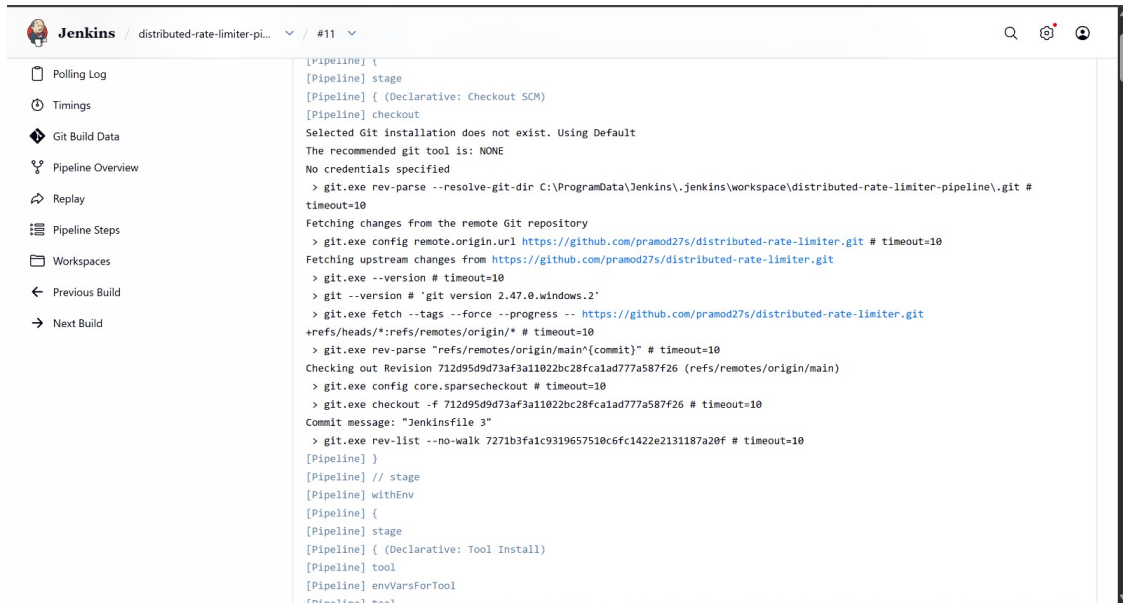
```

Figure 10.1: Jenkins CD Stage – Deployment Console Output and Port Binding Confirmation. The log shows the `JENKINS_NODE_COOKIE` variable being set to `dontKillMe`, the `start /B java -jar` command being executed, the stage completing with exit code 0, and the overall pipeline build status marked as `SUCCESS`.

The deployment console output confirms that the Spring Boot server was launched successfully as an independent background process. The `JENKINS_NODE_COOKIE` override is the critical detail in this log: without it, the Jenkins Process Tree Killer would terminate the server within seconds of the pipeline completing. Once this stage exits with code 0, the application is live and accepting requests on port 8080, connected to the Redis Cloud cluster.

11. Execution and Output Demonstration

11.1 Stage 1 – Port Cleanup Output

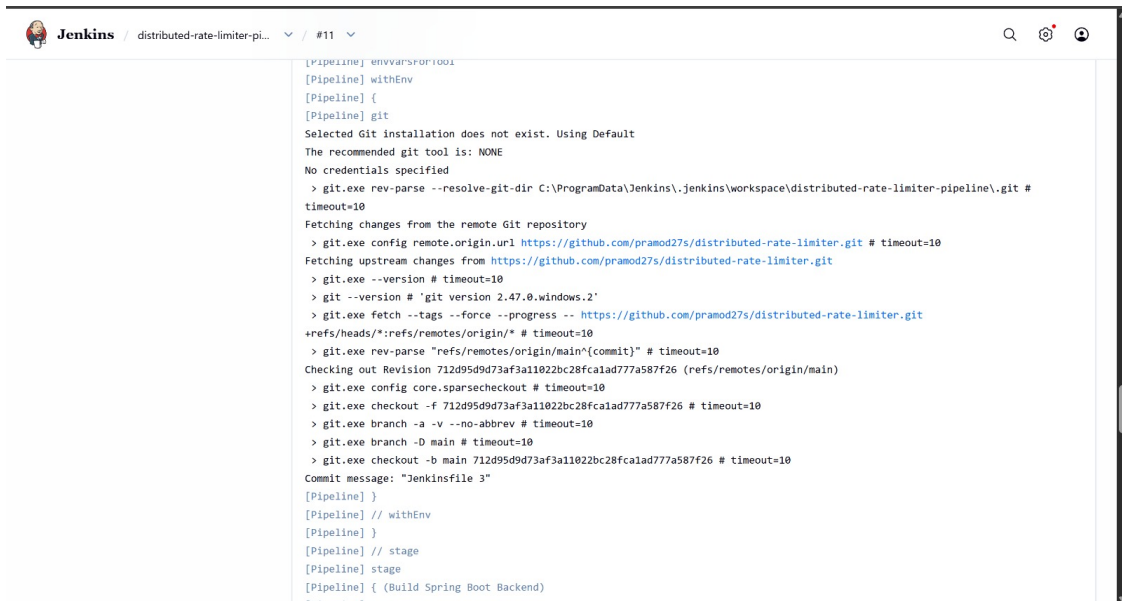


```
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\.git #
timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/pramod27s/distributed-rate-limiter.git # timeout=10
Fetching upstream changes from https://github.com/pramod27s/distributed-rate-limiter.git
> git.exe --version # timeout=10
> git --version # 'git version 2.47.0.windows.2'
> git.exe fetch --tags --force --progress -- https://github.com/pramod27s/distributed-rate-limiter.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 712d95d9d73af3a11022bc28fca1ad777a587f26 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 712d95d9d73af3a11022bc28fca1ad777a587f26 # timeout=10
Commit message: "Jenkinsfile 3"
> git.exe rev-list --no-walk 7271b3falc9319657510c6fc1422e2131187a20f # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] }
```

Figure 11.1: Jenkins Stage 1 – Port Cleanup Console Output (netstat and TaskKill). The `netstat -ano | findstr :8080` command locates the PID of any existing Java process currently bound to port 8080, followed by `TaskKill /PID <pid> /F` terminating it. If no process is found, the script exits cleanly with a zero error code, and the pipeline proceeds to Stage 2.

This output demonstrates the first line of defence against file-lock failures on Windows. By forcefully terminating the previously deployed server before the Maven build begins, the pipeline guarantees that the `target/` directory can be fully deleted by `mvn clean`. This ordering was the key architectural decision that resolved the `java.io.IOException` described in the Challenges section.

11.2 Stage 2 – Source Checkout Output



```

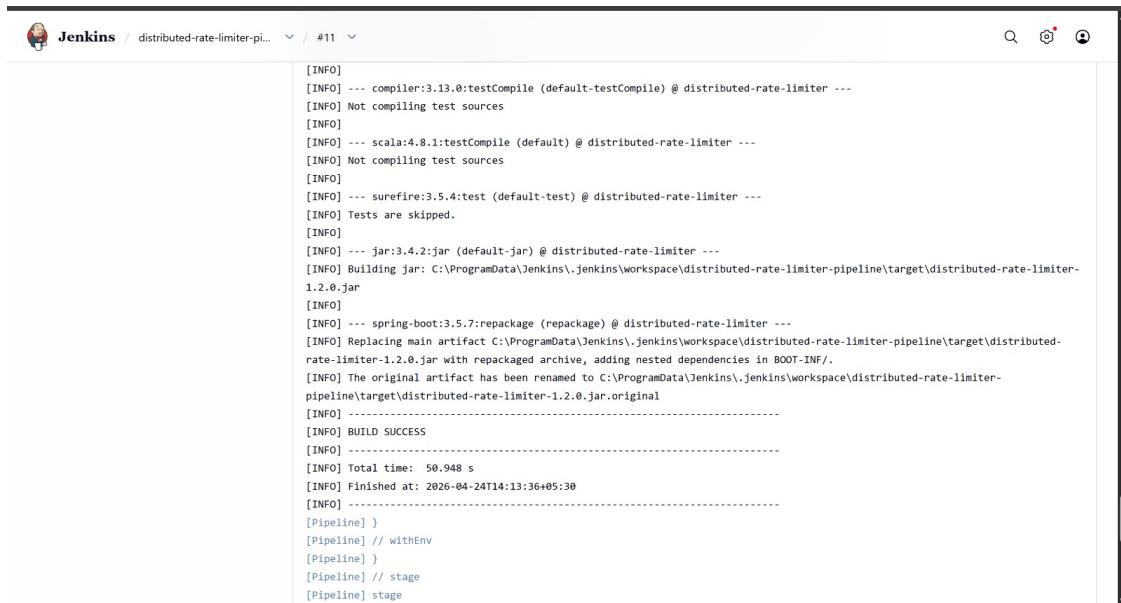
Jenkins / distributed-rate-limiter-pi... / #11
[Pipeline] envvarsForRoot
[Pipeline] withEnv
[Pipeline] {
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\distributed-rate-limiter-pipeline\.git #
timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url https://github.com/pramod27s/distributed-rate-limiter.git # timeout=10
Fetching upstream changes from https://github.com/pramod27s/distributed-rate-limiter.git
> git.exe --version # timeout=10
> git --version # 'git version 2.47.0.windows.2'
> git.exe fetch --tags --force --progress -- https://github.com/pramod27s/distributed-rate-limiter.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 712d95d9d73af3a11022bc28fca1ad777a587f26 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f 712d95d9d73af3a11022bc28fca1ad777a587f26 # timeout=10
> git.exe branch -a -v --no-abbrev # timeout=10
> git.exe branch -D main # timeout=10
> git.exe checkout -b main 712d95d9d73af3a11022bc28fca1ad777a587f26 # timeout=10
Commit message: "Jenkinsfile 3"
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Spring Boot Backend)
...

```

Figure 11.2: Jenkins Stage 2 – GitHub Source Checkout and Delta Resolution. The console log shows Jenkins authenticating to the GitHub repository, resolving the delta between the last-built commit and the latest `main` branch HEAD, and updating the local workspace. The SHA-1 commit hash of the checked-out revision is visible in the output.

The source checkout output confirms that the pipeline is always building from the exact commit that triggered it. The displayed commit hash ties the resulting JAR artefact to a specific, auditable point in the repository history, enabling reproducible builds and rollback if needed.

11.3 Stage 3 – Maven Build Output



```
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ distributed-rate-limiter ---
[INFO] Not compiling test sources
[INFO]
[INFO] --- scala:4.8.1:testCompile (default) @ distributed-rate-limiter ---
[INFO] Not compiling test sources
[INFO]
[INFO] --- surefire:3.5.4:test (default-test) @ distributed-rate-limiter ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ distributed-rate-limiter ---
[INFO] Building jar: C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\target\distributed-rate-limiter-1.2.0.jar
[INFO]
[INFO] --- spring-boot:3.5.7:repackage (repackage) @ distributed-rate-limiter ---
[INFO] Replacing main artifact C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\target\distributed-rate-limiter-1.2.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\ProgramData\Jenkins\jenkins\workspace\distributed-rate-limiter-pipeline\target\distributed-rate-limiter-1.2.0.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 50.948 s
[INFO] Finished at: 2026-04-24T14:13:36+05:30
[INFO] -----
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
```

Figure 11.3: Jenkins Stage 3 – Maven Build: Compile, Package, and BUILD SUCCESS. The Maven console log confirms dependency resolution from the local cache and Maven Central, Java 21 compilation with zero errors or warnings, and the final BUILD SUCCESS message. The log line showing `target/distributed-rate-limiter-1.2.0.jar` confirms the fat JAR artefact was assembled correctly.

The BUILD SUCCESS message is the gate that separates CI from CD. Only when Maven exits with code 0 does Jenkins proceed to Stage 4. A compilation error, an unresolved dependency, or a test failure (if enabled) would abort the pipeline here, preventing a broken artefact from ever being deployed to the runtime environment.

11.4 Stage 4 – Deployment and Application Running

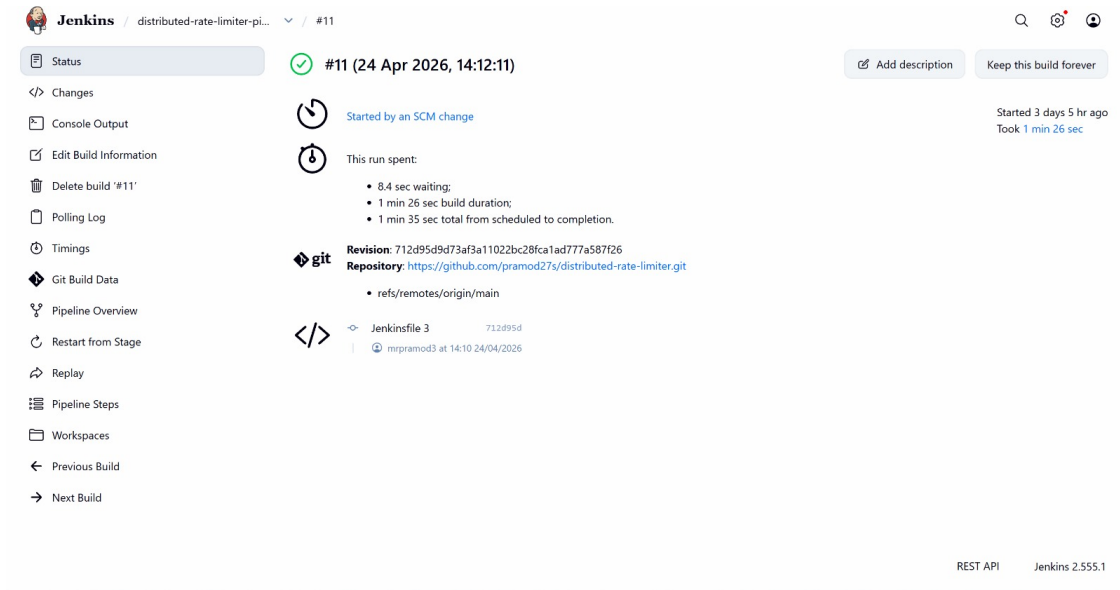


Figure 11.4: Jenkins Stage 4 – Background Deployment and Application Start. The console output confirms the `start /B java -jar` command was issued, the stage completed with exit code 0, and the overall pipeline build is marked as SUCCESS. The Spring Boot application is now running as an independent background process on port 8080.

At the point this stage exits, the updated Distributed Rate Limiter is live. The Spring Boot application context initialises in approximately 8–12 seconds after the `start /B` command returns, during which it establishes a connection pool to the Redis Cloud cluster and registers the `HandlerInterceptor` chain. All subsequent API requests are processed by the newly deployed version.

11.5 Full Pipeline Overview

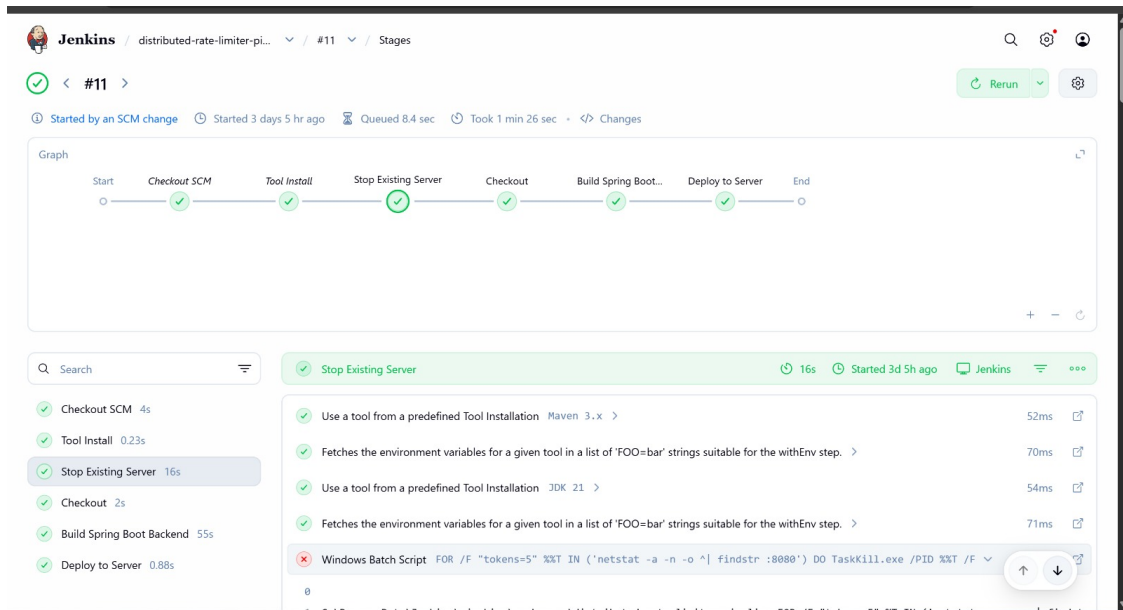


Figure 11.5: Jenkins Dashboard – Full Four-Stage Pipeline: All Stages Passed. The Blue Ocean or classic stage view shows all four pipeline stages in a horizontal bar, each marked with a green success indicator and its individual execution time. The total build duration and the SUCCESS status badge confirm the end-to-end CI/CD cycle completed without errors.

The full pipeline dashboard screenshot is the definitive proof of the complete CI/CD workflow operating correctly. From the moment a `git push` is detected, through port cleanup, source checkout, Maven build, and background deployment, every stage succeeds and the updated application is live within the two-minute SCM polling window. The build history below the stage view records every previous run, providing an auditable log of all deployments.

12. Challenges and Solutions

12.1 Challenge 1: Locked File Compilation Error (Windows I/O)

Problem Description

Maven's `clean` phase, which deletes the `target/` directory, repeatedly failed with a file-lock exception during early pipeline iterations. The root cause was that the Spring Boot application from the *previous* successful deployment maintained an exclusive OS-level read-lock on the `.jar` file inside `target/`. Windows prevents any process from deleting or overwriting a file that another process holds open, causing Maven to abort with a `java.io.IOException: Unable to delete file`.

Resolution

The Jenkinsfile stage ordering was restructured. The server shutdown logic (the `TaskKill` batch script) was moved to **Stage 1**, *before* the source checkout and Maven build stages. By terminating the running application at the very beginning of the pipeline, the file lock is released before Maven's `clean` phase ever attempts to access the directory. This ordering guarantees a clean build environment on every pipeline run.

Lesson learned: On Windows, resource locking is an OS-level concern that must be explicitly managed in pipeline design. Stage ordering is a critical architectural decision, not merely a stylistic one.

12.2 Challenge 2: Jenkins Process Tree Killer (Orphaned Child Jobs)

Problem Description

After the Maven build and deployment stage completed successfully, the newly started Spring Boot server terminated within seconds. Jenkins' internal **Process Tree Killer** was responsible. Upon pipeline completion, Jenkins sweeps the operating system for all child processes spawned during the build and terminates them to prevent memory leaks from abandoned builds. Because the Java server was launched as a child process of the Jenkins build agent, it was classified as a build artefact and automatically killed.

Resolution

The official Jenkins-recommended solution was implemented:

```
1 set JENKINS_NODE_COOKIE=dontKillMe
```

Listing 12.1: Jenkins Process Tree Killer Bypass – Environment Variable Override

What this does: Setting `JENKINS_NODE_COOKIE` to any value other than the default Jenkins build cookie modifies the environment tracking identifier of the spawned process. The Process Tree Killer identifies child processes by matching their `JENKINS_NODE_COOKIE` value against the current build's cookie. By overwriting it, the Java server is effectively “un-parented” from the

build and excluded from the cleanup sweep. The server persists indefinitely after the pipeline completes.

Lesson learned: Understanding the internal lifecycle management mechanisms of CI/CD tools is essential when deploying long-running background processes from within a pipeline.

13. Conclusion

13.1 Learnings from the DevOps Pipeline

This project provided comprehensive, hands-on experience with the full spectrum of modern DevOps practices:

- **Jenkins configuration:** Installing and configuring Jenkins as a Windows service, setting up Global Tool Configurations for JDK and Maven, creating a Declarative Pipeline job, and writing a production-quality `Jenkinsfile`.
- **Windows-specific DevOps challenges:** Diagnosing and resolving OS-level file locking and understanding Jenkins process tree management – both of which behave differently from equivalent Linux environments.
- **Infrastructure as Code:** Representing the complete deployment pipeline as a versioned `Jenkinsfile` in the source repository, making the pipeline reproducible, auditable, and team-shareable.
- **Systematic debugging:** Diagnosing two distinct production pipeline failures using console logs and error traces, then applying targeted, minimal fixes rather than workarounds.

13.2 Importance of CI/CD

The operational impact of this pipeline was immediately tangible during development. Every code change – whether a configuration adjustment, a bug fix, or a new feature – was automatically integrated and deployed within **two minutes** of the `git push` command. This eliminated all manual build-and-deploy overhead and ensured the running application was always synchronised with the latest codebase.

Continuous Integration enforces build-level quality gates, while Continuous Deployment reduces the lead time from code commit to running feature. Together, they embody the core DevOps principle of shortening the software delivery lifecycle while maintaining operational reliability. This project demonstrated that even on a local Windows development environment, a fully automated CI/CD pipeline is achievable with open-source tooling.

14. References

1. Redis Documentation. *Redis Commands Reference*. Redis Ltd., 2024. Available: <https://redis.io/docs/latest/commands/>
2. Redis Documentation. *Redis Data Types Overview*. Redis Ltd., 2024. Available: <https://redis.io/docs/latest/develop/data-types/>
3. Spring Boot Documentation. *Spring Boot Reference Guide, v3.5.x*. VMware Inc., 2024. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
4. Jenkins Documentation. *Jenkins User Handbook – Declarative Pipeline Syntax*. CloudBees Inc., 2024. Available: <https://www.jenkins.io/doc/book/pipeline/syntax/>
5. Apache Maven Project. *Maven Getting Started Guide*. Apache Software Foundation, 2024. Available: <https://maven.apache.org/guides/getting-started/>
6. Fowler, M. and Humble, J. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
7. Tanenbaum, A.S. and Van Steen, M. *Distributed Systems: Principles and Paradigms*, 3rd ed. Prentice Hall, 2017.
8. Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
9. Lettuce Project. *Lettuce – Advanced Java Redis Client*. Available: <https://lettuce.io/>
10. Visvesvaraya Technological University. *Scheme and Syllabus – B.E. Computer Science and Engineering (Data Science), 2022 Scheme*. VTU, 2022.